

MPI Parallelized Gaussian Process Regression for Efficient Frequency Interpolation for VWT-IE-Solver

Aiden Incheol Jung

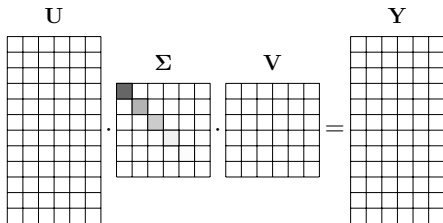
Department of Electric and Computer Engineering
University of Illinois Urbana-Champaign

incheol3@illinois.edu

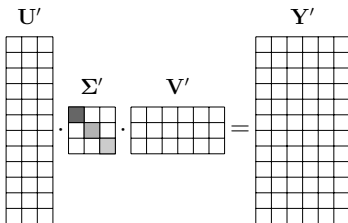
November 4, 2025

- VWT-IE-Solver harnesses capability of efficient angular interpolation using Principal Component Analysis (PCA).
- To extend this ability into the frequency axis, we employed a Gaussian Process Regression (GPR) with a Variance Minimizing Adaptive Sampler (VMAS).
- Running simulations on each new sample is “embarrassingly parallel,” allowing Message Passing Interface (MPI) protocol for parallel computing.

Theory: PCA



↓ Truncation based on Σ



- PCA is a representative technique for matrix dimension reduction.
- Singular Value Decomposition (SVD) is used for matrix analysis.

$$U_{N \times S} \Sigma_{S \times S} V_{S \times M}^T = Y_{N \times M}$$

- $Z := U\Sigma$ is defined as coefficients, whereas V^T functions as **basis**.
- Based on Σ , insignificant values are truncated:

$$V' = V[:, k] \quad \text{where, } \Sigma[k]/\Sigma[0] < \epsilon$$

- The matrix is then reconstructed: $Y = Z'V'^T$

Theory: GPR

- GPR is a widely-used technique for a surrogate modelling.
- For given training set $\mathcal{D} = \{(\mathbf{x}_n, \mathbf{y}_n = f(\mathbf{x}_n))\}_{n=1}^N$, where f refers to an expensive function that we aim to build a surrogate model for, the goal is to predict $\mathbf{y}_* = f(\mathbf{x}_*)$ where $\mathbf{x}_*$ is in testing set.

$$\begin{pmatrix} \mathbf{y} \\ \mathbf{y}_* \end{pmatrix} \sim \left(\begin{pmatrix} \boldsymbol{\mu} \\ \boldsymbol{\mu}_* \end{pmatrix}, \begin{pmatrix} \mathbf{K} & \mathbf{K}_* \\ \mathbf{K}_*^T & \mathbf{K}_{**} \end{pmatrix} \right)$$

- Kernel functions are used to model covariance between sampling points. It is the kernel function parameters that are trained for the given training data.

$$k(\mathbf{X}, \mathbf{X}) = \mathbf{K}, \quad k(\mathbf{X}, \mathbf{X}_*) = \mathbf{K}_*, \quad k(\mathbf{X}_*, \mathbf{X}_*) = \mathbf{K}_{**}$$

- The conditional posterior mean is analytically known as:

$$\begin{aligned} p(\mathbf{y}_* | \mathbf{X}_*, \mathbf{X}, \mathbf{y}) &= \mathcal{N}(\hat{\boldsymbol{\mu}}_*, \hat{\mathbf{K}}_{**}) \\ \hat{\boldsymbol{\mu}}_* &= \boldsymbol{\mu}(\mathbf{X}_*) + \mathbf{K}_*^T \mathbf{K}^{-1} (\mathbf{y} - \boldsymbol{\mu}(\mathbf{X})) \\ \hat{\mathbf{K}}_{**} &= \mathbf{K}_{**} - \mathbf{K}_*^T \mathbf{K}^{-1} \mathbf{K}_* \end{aligned}$$

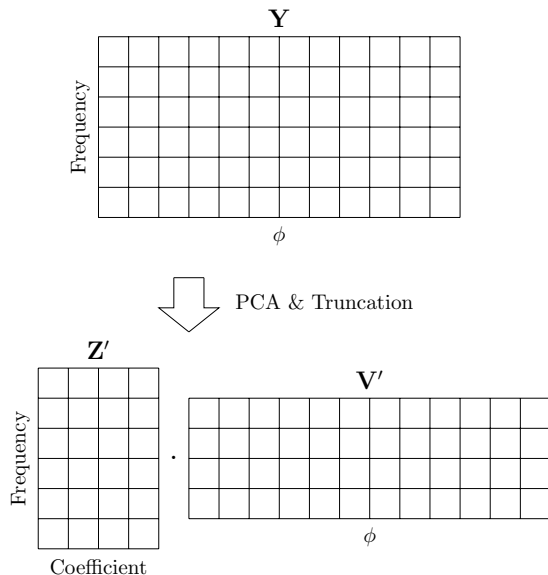
Theory: VMAS

- For GPR model to attain high accuracy, the training samples must effectively represent the behavior of the complete data.
- With expensive objective function such as a full-wave solver, samples are demanded to be small yet still highly representative.
- **Adaptive sampling** is one of the most popular way to enable this: it iteratively selects new samples, gradually revealing the data behavior.
- Considering K number of basis and Var_k being the variance from k th GPR, integrated variance can be calculated as:

$$\text{Var} = \sum_{k=1}^K \Sigma_k \text{Var}_k, \quad E = \text{Var} / \sum_{k=1}^K \Sigma_k$$

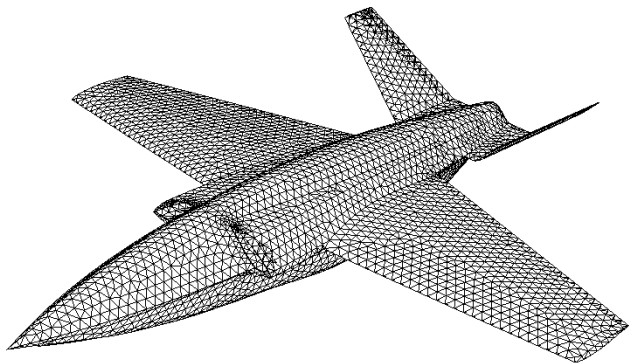
where E is referred to as the prediction energy fraction.

- VMAS selects frequencies with the largest variance: $f_{\text{next}} = \arg \max [\text{Var}(f)]$



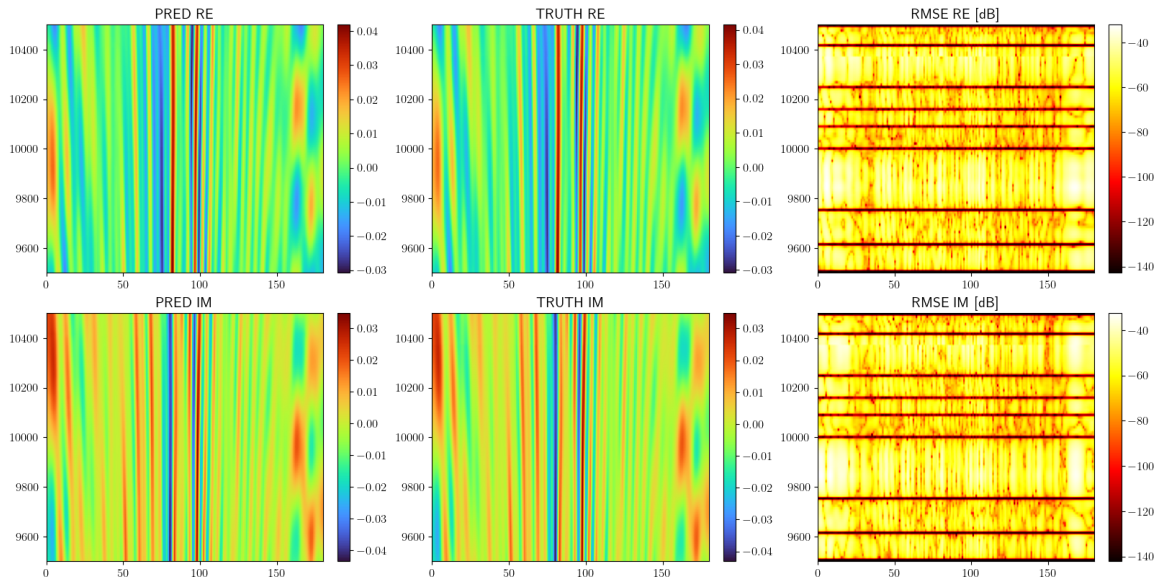
- Data of interest is 2D: [frequency, input angle].
- Direct prediction of the dataset is expensive and ineffective.
- PCA is employed to reduce the dataset, where the GPR model predicts the coefficient Z' .
- This **Reduced Basis** approach is also incorporated with **VMAS**, thus minimizing computational load while maintaining accuracy.

Demonstration 1: pre-computed farfields from PRIME aircraft



Total # Freq	9
RMSE	0.00065

Demonstration 1: pre-computed farfields from PRIME aircraft



Demonstration 2: on-fly simulation on a composite rounded cylinder

Mesh
DB: bottom_ext_current.silo
Cycle: 0
Var: mesh

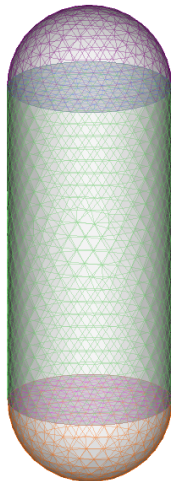
Mesh
DB: bottom_int_current.silo
Cycle: 0
Var: mesh

Mesh
DB: middle_ext_current.silo
Cycle: 0
Var: mesh

Mesh
DB: middle_int_current.silo
Cycle: 0
Var: mesh

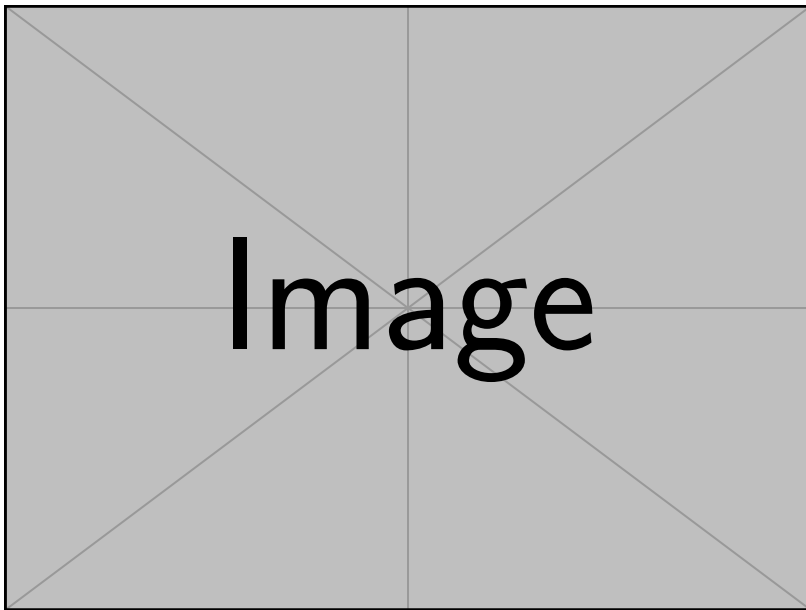
Mesh
DB: top_ext_current.silo
Cycle: 0
Var: mesh

Mesh
DB: top_int_current.silo
Cycle: 0
Var: mesh



Total # Freq RMSE

Demonstration 2: on-fly simulation on a composite rounded cylinder



Parallel GPR: workflow

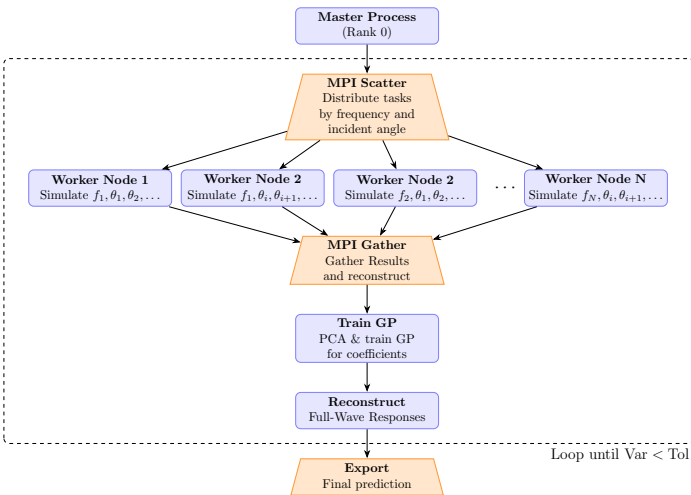


Figure: Workflow

- Master-worker pattern is used.
- Initial frequency samples are generated and distributed.
- Each worker node runs simulation.
- Simulation results are gathered in master node in which post-processed
- This loops until the normalized prediction energy is smaller than tolerance.
- When loop, master node samples the next frequencies to scatter across computing nodes.
- When terminates, master node exports the final prediction.

Parallel GPR: task chunking

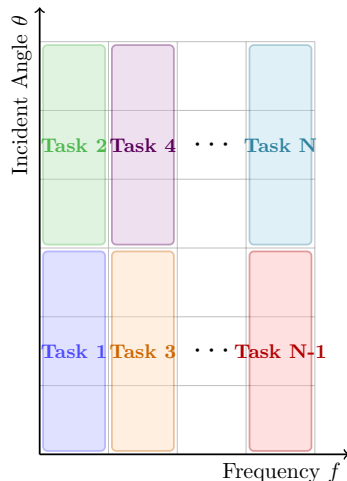


Figure: Workflow

- VWT-IE-Solver is a frequency-domain (FD) solver, naturally being embarrassingly parallel per frequency.
- In the sense of adaptive sampling, smaller number of added samples is encouraged to gradually reveal the data behavior.
- In general cluster environment, number of computing nodes are expected to be large.
- To grant the full computational capability, Each task is not only divided along the frequency axis but also the angular axis such that the number of tasks matches the number of nodes even when using a small number of added samples.

Demonstration 3: PRIME aircraft

Scaling study: Strong scaling

Demonstration 4: Tanker aircraft

Demonstration 5: Tanker aircraft

Conclusion

Appendix: GPR-kernels

- Radial basis function (RBF)

$$k_{\text{RBF}}(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right)$$

- Periodic

$$k_{\text{periodic}}(\mathbf{x}, \mathbf{x}') = \exp - \frac{2 \sin^2\left(\pi \sqrt{\|\mathbf{x} - \mathbf{x}'\|^2/p}\right)}{l^2}$$

- RBF-periodic (RBFP)

$$k_{\text{RBFP}}(\mathbf{x}, \mathbf{x}') = k_{\text{RBF}}(\mathbf{x}, \mathbf{x}') \times k_{\text{periodic}}(\mathbf{x}, \mathbf{x}')$$

- Stacked-RBFP

$$k_{\text{Stacked-RBFP}}^N(\mathbf{x}, \mathbf{x}') = \sum_{i=0}^N k_{\text{RBFP}}^i(\mathbf{x}, \mathbf{x}')$$